

Canopy - SCA #2

Canopy

HALBORN

Canopy - SCA #2 - Canopy

Prepared by:  HALBORN

Last Updated 07/11/2025

Date of Engagement: June 18th, 2025 - July 1st, 2025

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
14	0	0	2	4	8

TABLE OF CONTENTS

1. Introduction	
2. Assessment summary	
3. Test approach and methodology	
4. Caveat	
5. Risk methodology	
6. Scope	
7. Assessment summary & findings overview	
8. Findings & Tech Details	
8.1 Fee recipients cannot access earned fees in meridian strategy	
8.2 Router's meridian strategy deposit path mismatch makes strategy inaccessible	
8.3 Reward notification lacks base asset exclusion check	
8.4 Incorrect balance query in layerbank airdrop block's claim_behalf function	
8.5 Inconsistent error codes across strategy implementations	
8.6 Permanent upkeep interval limitation in layerbank and echelon strategies	
8.7 Missing validation of rewards pool address	
8.8 Missing validation of upkeep interval thresholds	
8.9 Redundant state update in initial rewards pool assignment	
8.10 Missing zero amount validation in echelon strategy vault deposit functions	

8.11 Documentation inconsistencies in strategy comments

8.12 Unused code

8.13 Unnecessary variable assignments

8.14 Inefficient global borrow of strategy resource in rewards pool setter

1. Introduction

Canopy engaged **Halborn** to conduct a security assessment of some packages for **Movement** blockchain, beginning on June 18th, 2025, and ending on July 1st, 2025. This security assessment focused on the smart contracts within the [Satay-movement GitHub](#) repository, commit hashes, and further details can be found in the Scope section of this report. The primary focus of this audit is to review the new Meridian strategy and a newly implemented rewards notification feature within the LayerBank and Echelon strategies.

Canopy is a yield aggregator on the Movement network that enables efficient interaction with multiple DeFi protocols through modular strategies composed of predefined operations. It also provides automated vaults that optimize fund allocations and rebalance strategy debt ratios, offering a streamlined approach to yield generation with reduced user intervention.

2. Assessment Summary

The team at Halborn assigned a full-time security engineer to verify the security of the smart contracts. The security engineer is a blockchain and smart-contract security expert with advanced penetration testing, smart-contract hacking, and deep knowledge of multiple blockchain protocols.

The purpose of this assessment is to:

- Ensure that smart contract functions operate as intended.
- Identify potential security issues with the smart contracts.

In summary, Halborn identified some improvements to reduce the likelihood and impact of risks, which were mostly addressed by the **Canopy team**. The main ones were the following:

- Make the `withdraw_fa` function publicly accessible in the Meridian strategy and add proper authorization checks to allow fee recipients to withdraw their strategy shares.
- Create a deposit function that allows Meridian strategy deposits to be handled through the appropriate FA path while supporting the required `CoinType` parameter.
- Add validation in the rewards notification functions to skip processing when the reward asset matches the strategy's base asset.
- Fix the `claim_behalf` function to query balances from the `on_behalf_of` address instead of the signer's address when calculating claimed rewards.

3. Test Approach And Methodology

Halborn performed a combination of the manual view of the code and automated security testing to balance efficiency, timeliness, practicality, and accuracy regarding the scope of the smart contract assessment. While manual testing is recommended to uncover flaws in logic, process, and implementation, automated testing techniques help enhance the coverage of smart contracts. They can quickly identify items that do not follow security best practices. The following phases and associated tools were used throughout the term of the assessment:

- Research into the architecture, purpose, and use of the platform.
- Manual code review and walkthrough.
- Manual assessment of the critical Move variables and functions in scope to identify any vulnerability classes related to arithmetic or logic.
- Cross-contract call controls.
- Logical controls related to the platform architecture.
- Integration testing using the Aptos Framework.

4. Caveat

While the audit team conducted a thorough static review and manual analysis, dynamic validation (e.g., automated test execution or proof-of-concept reproduction) could not be performed due to technical constraints outside the scope of this engagement. Specifically, a build error caused by third-party package dependencies prevented the execution of the existing test suite within the audit environment.

As a result, end-to-end tests could not be executed, and certain behaviors could not be verified through runtime instrumentation. Although this limited the ability to validate some edge cases dynamically, the findings presented are based on careful source code analysis and reflect the issues observable through that approach.

5. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

5.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

5.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

Impact Metric (M_I)	Metric Value	Numerical Value
Confidentiality (C)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

5.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (<i>C</i>)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (<i>r</i>)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (<i>s</i>)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4

SEVERITY	SCORE VALUE RANGE
Informational	0 - 1.9

6. SCOPE

FILES AND REPOSITORY
(a) Repository: satay-movement (b) Assessed Commit ID: 88b2d60 (c) Items in scope: - packages/strategies/echelon_simple/sources/strategy.rs - packages/strategies/layerbank_simple/sources/strategy.rs - packages/strategies/meridian_rewards/sources/strategy.rs - packages/blocks/meridian_farming/sources/meridian_farming_block.rs - packages/blocks/layerbank_airdrop/sources/layerbank_airdrop_block.rs - packages/router/sources/router.move - packages/router/sources/auth.move - packages/router/sources/harvest.move - packages/router/sources/deposit.move - packages/router/sources/withdraw.move
Out-of-Scope: Third party dependencies and economic attacks.
REMEDIATION COMMIT ID:
https://github.com/Canopyxyz/satay-movement/pull/72
Out-of-Scope: New features/implementations after the remediation commit IDs.

7. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
0	0	2	4	8

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
FEE RECIPIENTS CANNOT ACCESS EARNED FEES IN MERIDIAN STRATEGY	MEDIUM	SOLVED - 07/09/2025
ROUTER'S MERIDIAN STRATEGY DEPOSIT PATH MISMATCH MAKES STRATEGY INACCESSIBLE	MEDIUM	SOLVED - 07/09/2025
REWARD NOTIFICATION LACKS BASE ASSET EXCLUSION CHECK	LOW	SOLVED - 07/07/2025
INCORRECT BALANCE QUERY IN LAYERBANK AIRDROP BLOCK'S CLAIM_BEHALF FUNCTION	LOW	SOLVED - 07/07/2025
INCONSISTENT ERROR CODES ACROSS STRATEGY IMPLEMENTATIONS	LOW	RISK ACCEPTED - 07/07/2025
PERMANENT UPKEEP INTERVAL LIMITATION IN LAYERBANK AND ECHELON STRATEGIES	LOW	SOLVED - 07/07/2025
MISSING VALIDATION OF REWARDS POOL ADDRESS	INFORMATIONAL	SOLVED - 07/09/2025
MISSING VALIDATION OF UPKEEP INTERVAL THRESHOLDS	INFORMATIONAL	SOLVED - 07/07/2025
REDUNDANT STATE UPDATE IN INITIAL REWARDS POOL ASSIGNMENT	INFORMATIONAL	SOLVED - 07/07/2025

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
MISSING ZERO AMOUNT VALIDATION IN ECHELON STRATEGY VAULT DEPOSIT FUNCTIONS	INFORMATIONAL	SOLVED - 07/07/2025
DOCUMENTATION INCONSISTENCIES IN STRATEGY COMMENTS	INFORMATIONAL	SOLVED - 07/07/2025
UNUSED CODE	INFORMATIONAL	SOLVED - 07/07/2025
UNNECESSARY VARIABLE ASSIGNMENTS	INFORMATIONAL	SOLVED - 07/07/2025
INEFFICIENT GLOBAL BORROW OF STRATEGY RESOURCE IN REWARDS POOL SETTER	INFORMATIONAL	SOLVED - 07/07/2025

8. FINDINGS & TECH DETAILS

8.1 FEE RECIPIENTS CANNOT ACCESS EARNED FEES IN MERIDIAN STRATEGY

// MEDIUM

Description

The **Meridian** strategy's `withdraw_fa` function is declared as `public(friend)`, but the only friend modules are test modules (`meridian_rewards::withdraw_fa_test_entry_functions` and `meridian_rewards::deposit_fa_test_entry_functions`). This effectively makes the function inaccessible in production, unlike the **Echelon** and **LayerBank** strategies which have `public entry` functions.

The `withdraw_fa` function is designed to allow fee recipients (strategy managers and protocol fee recipients) to withdraw their strategy shares and redeem them for the base asset.

However, since the Meridian strategy's `withdraw_fa` function is not publicly accessible, fee recipients cannot withdraw their accumulated strategy shares. As a result, fees are effectively distributed but unrecoverable, causing the strategy to operate without generating any actual profit for its intended beneficiaries.

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:M/I:N/D:N/Y:C (5.6)

Recommendation

It is recommended to:

- Change the `withdraw_fa` function visibility from `public(friend)` to `public entry` in the **Meridian** strategy, following the same pattern as the **Echelon** and **LayerBank** strategies.
- Add authorization checks via `ensure_fee_recipient` to validate that only authorized fee recipients can withdraw their strategy shares.

This will ensure that fee recipients can properly withdraw their strategy shares and redeem them for the base asset, while maintaining security through proper access controls.

Remediation Comment

SOLVED : The `Canopy team` solved this issue in the specified pull request.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/pull/72>

8.2 ROUTER'S MERIDIAN STRATEGY DEPOSIT PATH MISMATCH MAKES STRATEGY INACCESSIBLE

// MEDIUM

Description

The router exhibits a major architectural inconsistency in how it handles deposits for the **Meridian** strategy. The Meridian strategy requires a `CoinType` parameter for deposits as it operates with wrapped staking coins, but the router incorrectly calls `meridian_rewards::vault_deposit_fa<CoinType>` from the `deposit_coin` (coin path) instead of `deposit_fa` (FA path).

This creates a fundamental mismatch between the router's expected flow and the **Meridian** strategy's actual requirements:

1. **Router expects:** Users to deposit wrapper coins through `deposit_coin`, which are then unwrapped to extract fungible assets
2. **Meridian reality:** Users cannot hold wrapped coins directly, and the strategy expects fungible assets to be deposited and then wrapped internally

Since deposits are broken, the withdrawal functionality is irrelevant as users cannot deposit into the strategy in the first place. However, it's worth noting that the withdrawal side does have proper support through `withdraw_fa_with_coin_type` for strategies like **Meridian** that require coin type parameters, while the deposit side lacks an equivalent `deposit_fa_with_coin_type` function.

Code Location

Below is the implementation of the `deposit_coin` function:

```
22 public(friend) fun deposit_coin<CoinType>(  
23     vault: Object<Vault>,  
24     strategy: Object<BaseStrategy>,  
25     packet: Option<vector<u8>>,  
26     amount: u64  
27 ) {  
28     let borrowed_router_ref = auth::borrow_router_ref();  
29     let vault_signer = vault::get_signer_for_router(vault, auth::borrowed_ref(&borrowed_router_ref));  
30     auth::return_router_ref(borrowed_router_ref);  
31  
32     let concrete_address = base_strategy::concrete_address(strategy);  
33     if (concrete_address == @echelon_simple) {  
34         echelon_simple::vault_deposit_coin<CoinType>(&vault_signer, strategy, amount);  
35     } else if (concrete_address == @moveposition_simple) {  
36         let packet = option::destroy_some(packet);  
37         moveposition_simple::vault_deposit_coin<CoinType>(&vault_signer, strategy, packet, amount);  
38     } else if (concrete_address == @meridian_rewards) {  
39         meridian_rewards::vault_deposit_fa<CoinType>(&vault_signer, strategy, amount);  
40     }  
41 }
```

Below is the implementation of the `deposit_fa` function:


```

43 public(friend) fun deposit_fa(
44     vault: Object<Vault>,
45     strategy: Object<BaseStrategy>,
46     packet: Option<vector<u8>>,
47     amount: u64
48 ) {
49     let borrowed_router_ref = auth::borrow_router_ref();
50     let vault_signer = vault::get_signer_for_router(vault, auth::borrowed_ref(&borrowed_router_ref));
51     auth::return_router_ref(borrowed_router_ref);
52
53     let concrete_address = base_strategy::concrete_address(strategy);
54     if (concrete_address == @echelon_simple) {
55         echelon_simple::vault_deposit_fa(&vault_signer, strategy, amount);
56     } else if (concrete_address == @moveposition_simple) {
57         let packet = option::destroy_some(packet);
58         moveposition_simple::vault_deposit_fa(&vault_signer, strategy, packet, amount);
59     } else if (concrete_address == @layerbank_simple) {
60         layerbank_simple::vault_deposit_fa(&vault_signer, strategy, amount);
61     };
62 }

```

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:C/I:M/D:N/Y:N (5.6)

Recommendation

It is recommended to create a `deposit_fa_with_coin_type` function that mirrors the withdrawal pattern, allowing **Meridian** strategy deposits to be handled through the appropriate FA path while supporting the required `CoinType` parameter. This will ensure consistent architectural patterns between deposit and withdrawal operations for strategies that require coin type parameters.

Remediation Comment

SOLVED: The **Canopy team** solved this issue in the specified pull request.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/pull/72>

8.3 REWARD NOTIFICATION LACKS BASE ASSET EXCLUSION CHECK

// LOW

Description

Several strategy implementations fail to check whether reward assets are different from the strategy's base asset before notifying rewards. This can lead to mistakenly notifying rewards in the base asset, which should be skipped.

This affects the following functions:

1. Meridian Strategy:

- `notify_rewards`

2. LayerBank Strategy:

- `notify_rewards`
- `notify_all_rewards`

3. Echelon Strategy:

- `notify_rewards_coin`

Without proper validation, these functions may incorrectly notify rewards in the base asset, potentially leading to unexpected or incorrect reward distributions.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:M/D:N/Y:N (3.4)

Recommendation

It is recommended to add base asset validation in the aforementioned notify functions to ensure they only process rewards that are different from the strategy's base asset.

Remediation Comment

SOLVED : The `Canopy team` solved this issue in the specified pull request.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/pull/72>

8.4 INCORRECT BALANCE QUERY IN LAYERBANK AIRDROP BLOCK'S CLAIM_BEHALF FUNCTION

// LOW

Description

The `claim_behalf` function in the `LayerBank` airdrop block contains a logic error in its reward calculation. When checking balances before and after claiming, it queries the signer's account balance instead of the `on_behalf_of` address that actually receives the rewards. As a result, when these addresses differ, the function will incorrectly report zero rewards claimed even though rewards were successfully transferred to the beneficiary account. This creates a discrepancy between the actual rewards claimed and the amount reported by the function.

Currently, this risk is limited since `claim_behalf` is only invoked internally by the `claim` function and no strategies call it directly. However, the risk could increase if future strategies invoke `claim_behalf` directly or if the protocol expands to support claiming rewards on behalf of other addresses.

Code Location

Below is the implementation of the `claim_behalf` function:

```
49 | public fun claim_behalf(account: &signer, on_behalf_of: address): u64 {  
50 |     let pre_balance = primary_store_balance(signer::address_of(account), reward_metadata());  
51 |     simple_airdrop::claim_behalf(account, on_behalf_of);  
52 |     let post_balance = primary_store_balance(signer::address_of(account), reward_metadata());  
53 |     post_balance - pre_balance  
54 | }
```

Below is the implementation of the `claim` function:

```
45 | public fun claim(account: &signer): u64 {  
46 |     claim_behalf(account, signer::address_of(account))  
47 | }
```

BVSS

AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:H/D:N/Y:N (2.5)

Recommendation

It is recommended to modify the `claim_behalf` function to use the `on_behalf_of` address when querying balances, rather than the signer's address.

Remediation Comment

SOLVED : The `Canopy team` solved this issue in the specified pull request.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/pull/72>

8.5 INCONSISTENT ERROR CODES ACROSS STRATEGY IMPLEMENTATIONS

// LOW

Description

The three strategy implementations (**Echelon**, **Layerbank**, and **Meridian**) have inconsistent error code values for shared error types. For example:

- **EINVALID_FEE_RECIPIENT** : Echelon (11) vs Layerbank (9)
- **ENO_REWARDS_POOL_ADDRESS** : Echelon (12) vs Layerbank (10) vs Meridian (11)
- **EUPKEEP_NOT_READY** : Echelon (13) vs Layerbank (11) vs Meridian (13)
- **EINVALID_REWARDS_POOL_ADDRESS** : Echelon (14) vs Layerbank (12) vs Meridian (10)

While this does not affect core functionality since error handling happens within the protocol, it makes the code harder to maintain and debug. External systems that integrate with multiple strategies need to handle different error codes for the same error conditions, which increases complexity and the chance of bugs.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (2.5)

Recommendation

It is recommended to create a shared error constants module that all strategies can import to standardize error codes across the protocol and ensure consistent error handling.

Remediation Comment

RISK ACCEPTED : The **Canopy team** accepted the risk of this finding. They stated the following:
| Acknowledged. We decided to maintain independent error code lists. There is no impact on functionality.

8.6 PERMANENT UPKEEP INTERVAL LIMITATION IN LAYERBANK AND ECHELON STRATEGIES

// LOW

Description

The **LayerBank** and **Echelon** strategy implementations lack the ability to update the upkeep interval after strategy creation, unlike the **Meridian** strategy which includes a `set_upkeep_interval` function. Both **LayerBank** and **Echelon** strategies use a hardcoded `DEFAULT_UPKEEP_INTERVAL` of 24 hours (60*60*24 seconds) that cannot be modified once the strategy is deployed.

This creates a permanent operational limitation where strategy managers cannot adjust the frequency of reward distribution after strategy creation. The fixed 24-hour interval cannot be modified without deploying new strategy contracts, which affects the protocol's ability to adapt to changing market conditions and user preferences. This limitation is irreversible and creates inconsistency across strategy implementations.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

Recommendation

It is recommended to add a `set_upkeep_interval` function to both **LayerBank** and **Echelon** strategies, following the same pattern as the **Meridian** strategy implementation. Additionally, consider adding validation to ensure the new interval falls within reasonable bounds (e.g., minimum 1 hour, maximum 7 days) to prevent abuse and ensure operational efficiency.

Remediation Comment

SOLVED : The **Canopy team** solved this issue in the specified pull request.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/pull/72>

8.7 MISSING VALIDATION OF REWARDS POOL ADDRESS

// INFORMATIONAL

Description

The **Meridian** strategy's `create` and `set_rewards_pool_address` functions do not validate whether the provided `rewards_pool_address` (when not `None`) actually contains a `StakingPool` resource. Additionally, in both the **LayerBank** and **Echelon** strategies, the `set_rewards_pool_address` function exhibits inconsistent validation logic—if the strategy integrates a rewards system, it assigns the new address without verifying that it references a `StakingPool` resource, whereas strategies without rewards perform this validation.

This inconsistency allows strategies to be configured with invalid rewards pool addresses, which can lead to runtime failures when the strategy attempts to utilize the rewards functionality.

This issue occurs because the `get_staking_pool` function calls `object::address_to_object<StakingPool>(option::destroy_some(rewards_pool_address))`, which reverts with `ERESOURCE_DOES_NOT_EXIST` (error code 7) if a `StakingPool` resource does not exist at the specified address.

This vulnerability impacts multiple critical functions, including `notify_rewards`, `notify_all_rewards`, and `perform_upkeep`, thereby preventing proper reward distribution and automated upkeep mechanisms from functioning correctly.

Code Location

Below is the implementation of the `get_staking_pool` function:

```
24 | fun get_staking_pool(strategy: Object<BaseStrategy>): Object<StakingPool> acquires RewardsPoolDetails
25 |     let rewards_pool_address = rewards_pool_address(strategy);
26 |     assert!(option::is_some(&rewards_pool_address), ENO_REWARDS_POOL_ADDRESS);
27 |     object::address_to_object<StakingPool>(option::destroy_some(rewards_pool_address))
28 | }
```

Below is the implementation of the `notify_reward_amount_internal` function:

```
10 | fun notify_reward_amount_internal(
11 |     strategy_signer: &signer,
12 |     strategy: Object<BaseStrategy>,
13 |     reward_metadata: Object<Metadata>,
14 |     reward_amount: u64
15 | ) acquires RewardsPoolDetails {
16 |     // If a rewards pool address is set, notify the rewards module
17 |     let rewards_pool_address = rewards_pool_address(strategy);
18 |     if (option::is_some(&rewards_pool_address)) {
19 |         let staking_pool = get_staking_pool(strategy);
20 |         multi_rewards::notify_reward_amount(strategy_signer, staking_pool, reward_metadata, reward_amount);
21 |     }
22 | }
```

Below is a code snippet of the `notify_all_rewards_internal` function:

```

86 | fun notify_all_rewards_internal<WrapperStakeCoin>(
87 |     strategy: Object<BaseStrategy>
88 | ) acquires MeridianRewardsStrategy, RewardsPoolDetails {
89 |     let strategy_ref = borrow_strategy<WrapperStakeCoin>(strategy);
90 |     let strategy_signer = &base_strategy::get_signer(&strategy_ref.auth_ref);
91 |
92 |     let staking_pool = get_staking_pool(strategy);
93 |     // rest of the code
94 | }

```

BVSS

AO:S/AC:L/AX:L/R:P/S:U/C:N/A:C/I:M/D:N/Y:H (1.3)

Recommendation

It is recommended to add validation in both the `create` and `set_rewards_pool_address` functions for the **Meridian** strategy, and standardize the validation in `set_rewards_pool_address` for **LayerBank** and **Echelon** strategies, to ensure that the provided `rewards_pool_address` holds a valid `StakingPool` resource before storing it. This will prevent strategies from being created or configured with invalid rewards pool addresses that would cause runtime failures.

Remediation Comment

SOLVED: The `Canopy team` solved this issue in the specified pull request.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/pull/72>

8.8 MISSING VALIDATION OF UPKEEP INTERVAL THRESHOLDS

// INFORMATIONAL

Description

The `set_upkeep_interval` function in the **Meridian** strategy lacks validation to ensure that the new upkeep interval falls within appropriate lower and upper thresholds. Currently, the function only validates that the caller is the strategy manager but does not check if the provided `upkeep_interval` value is reasonable.

This allows strategy managers to set extremely short intervals (e.g., 1 second) which could lead to excessive gas consumption and potential DoS attacks through rapid upkeep calls, or extremely long intervals (e.g., 1 year) which could prevent timely reward distribution and maintenance operations.

The function should enforce minimum and maximum bounds for the upkeep interval to ensure:

- Minimum interval prevents excessive gas consumption and spam
- Maximum interval ensures regular maintenance and reward distribution
- Reasonable defaults align with protocol expectations

Code Location

Below is the implementation of the `set_upkeep_interval` function:

```
14 public entry fun set_upkeep_interval(  
15     account: &signer, strategy: Object<BaseStrategy>, upkeep_interval: u64  
16 ) acquires RewardsPoolDetails {  
17     assert!(  
18         signer::address_of(account) == base_strategy::manager(strategy),  
19         ENOT_AUTHORIZED  
20     );  
21  
22     let rewards_pool_details_ref = borrow_mut_rewards_pool_details(strategy);  
23     rewards_pool_details_ref.upkeep_interval = upkeep_interval;  
24 }
```

BVSS

AO:S/AC:L/AX:L/R:P/S:U/C:N/A:M/I:M/D:N/Y:C (1.3)

Recommendation

It is recommended to add validation in the `set_upkeep_interval` function to ensure the new interval falls within acceptable bounds.

Remediation Comment

SOLVED: The **Canopy team** solved this issue in the specified pull request by enforcing the upkeep interval to be between 1 hour and 7 days.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/pull/72>

8.9 REDUNDANT STATE UPDATE IN INITIAL REWARDS POOL ASSIGNMENT

// INFORMATIONAL

Description

The `set_rewards_pool_address` function in both **LayerBank** and **Echelon** strategies contains a redundant state update when initializing rewards pool details for strategies created before the **multi-rewards** package was integrated.

The issue occurs in the following logic flow:

1. If `RewardsPoolDetails` does not exist, the function invokes `initialize_reward_pool_details()` to create the resource with the specified `rewards_pool_address`.
2. Immediately afterward, the function unconditionally updates the `rewards_pool_address` field again, despite it having just been set during initialization.

This redundant update results in unnecessary gas consumption by performing the same state write operation twice. The second update is entirely superfluous, as it writes the same value that was just assigned during initialization.

Code Location

Below is the implementation of the `set_rewards_pool_address` function:

```
001 public entry fun set_rewards_pool_address(  
002     account: &signer, strategy: Object<BaseStrategy>, rewards_pool_address: Option<address>  
003 ) acquires RewardsPoolDetails, EchelonStrategy {  
004     assert!(  
005         signer::address_of(account) == base_strategy::manager(strategy),  
006         ENOT_AUTHORIZED  
007     );  
008  
009     // Strategies created before the multirewards package was integrated will not have a rewards pool  
010     // In this case, we initialize the rewards pool details.  
011     let strategy_ref = borrow_strategy(strategy);  
012     if (!exists<RewardsPoolDetails>(object::object_address(&strategy_ref))) {  
013         initialize_reward_pool_details(strategy_ref, rewards_pool_address);  
014     };  
015  
016     let rewards_pool_details_ref = borrow_mut_rewards_pool_details(strategy_ref);  
017     rewards_pool_details_ref.rewards_pool_address = rewards_pool_address;  
018 }
```

Below is the implementation of the `initialize_reward_pool_details` function:

```
050 fun initialize_reward_pool_details(  
051     strategy_ref: &EchelonStrategy, rewards_pool_address: Option<address>  
052 ) {  
053     let strategy_signer = base_strategy::get_signer(&strategy_ref.auth_ref);  
054     if (option::is_some(&rewards_pool_address)) {  
055         assert!(  
056             object::object_exists<StakingPool>(*option::borrow(&rewards_pool_address)),  
057             EINVALID_REWARDS_POOL_ADDRESS  
058         );  
059     }
```

```

59     };
60
61     move_to(
62         &strategy_signer,
63         RewardsPoolDetails {
64             rewards_pool_address,
65             last_upkeep_timestamp: 0,
66             upkeep_interval: DEFAULT_UPKEEP_INTERVAL
67         }
68     );
69 }

```

BVSS

AO:A/AC:L/AX:M/R:P/S:U/C:N/A:N/I:L/D:N/Y:N (0.8)

Recommendation

It is recommended to place the rewards pool address assignment logic inside an `else` clause to ensure the address is only updated when a change is actually required, avoiding the redundant state update during initial resource creation.

Remediation Comment

SOLVED : The `Canopy team` solved this issue in the specified pull request.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/pull/72>

8.10 MISSING ZERO AMOUNT VALIDATION IN ECHELON STRATEGY VAULT DEPOSIT FUNCTIONS

// INFORMATIONAL

Description

The Echelon strategy's `vault_deposit_coin` and `vault_deposit_fa` functions lack validation to ensure the deposit amount is greater than zero. Although the documentation comments specify that these functions should revert with `EINVALID_AMOUNT` if the amount is zero, the actual implementation does not perform this check.

This discrepancy between the documented behavior and the implementation allows for the possibility of processing zero amounts through the vault debt creation mechanism, resulting in potentially unnecessary or wasted operations.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to add the zero amount validation check at the start of both `vault_deposit_coin` and `vault_deposit_fa` functions in the Echelon strategy.

Remediation Comment

SOLVED : The `Canopy team` solved this issue in the specified pull request. The team stated the following:

The zero-amount validation is done on the vault level. Since the `vault_deposit_fa` and `vault_deposit_coin` functions can only be called by the vault, the check is not needed in these functions. We have updated the comment, which caused this confusion.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/pull/72>

8.11 DOCUMENTATION INCONSISTENCIES IN STRATEGY COMMENTS

// INFORMATIONAL

Description

Multiple strategy implementations contain incorrect documentation in their comments:

1. Protocol Name Inconsistencies:

- **LayerBank** Strategy:

```
92 | // this is the latest(i.e. now) base asset value of the strategy in the underlying echelon market
```

The comment incorrectly references "echelon market" instead of "layerbank market".

- **Meridian** Strategy:

```
67 | // we get the idle assets in the strategy and deploy it to LayerBank
```

The comment incorrectly references "LayerBank" instead of "Meridian".

2. Incorrect Error Description:

- **Echelon** Strategy:

```
85 | /// @reverts EINSUFFICIENT_BALANCE if the account does not have enough balance to deposit.
```

The comment incorrectly describes the error condition. In vault deposit functions, the signer's account is not depositing funds - the vault provides assets from its idle asset pool.

These documentation inconsistencies could confuse developers reading the code about which protocols the strategies interact with and how error conditions work.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Correct the comments to accurately reflect the underlying protocols and error conditions:

- **LayerBank** strategy: Change "echelon market" to "layerbank market".
- **Meridian** strategy: Change "LayerBank" to "Meridian".
- **Echelon** strategy: Update the error description to reference **EVAULT_INSUFFICIENT_ASSETS** when the vault lacks sufficient idle assets.

Remediation Comment

SOLVED : The Canopy team solved this issue in the specified pull request.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/pull/72>

8.12 UNUSED CODE

// INFORMATIONAL

Description

Several modules within the codebase contain unused code elements that should be removed to enhance code clarity and facilitate ongoing maintenance:

- **Unused Import** - LayerBank Strategy:

```
3 | use std::vector;
```

- **Unused Import** - LayerBank Airdrop Block:

```
6 | use aptos_framework::aptos_coin::{Self, AptosCoin};
```

- **Unused Variable** - LayerBank Strategy:

```
102 | let rewards_pool_address = *option::borrow(&pool_details_ref.rewards_pool_address);
```

- **Unused Variable** - LayerBank Strategy:

```
148 | let strategy_signer = &base_strategy::get_signer(&strategy_ref.auth_ref);
```

The detected issues involve imports that are declared but not utilized within their respective modules, as well as variables that are assigned values but not referenced thereafter.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to remove unused code segments to improve code maintainability and clarity:

- Remove `use std::vector;` from the LayerBank strategy implementation.
- Remove `Self` from the `aptos_coin` import statement in the LayerBank airdrop block module.
- Remove the unused `rewards_pool_address` variable assignment within the `harvest` function.
- Remove the unused `strategy_signer` variable assignment within the `notify_all_rewards_internal` function.

Remediation Comment

SOLVED : The Canopy team solved this issue in the specified pull request.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/pull/72>

8.13 UNNECESSARY VARIABLE ASSIGNMENTS

// INFORMATIONAL

Description

Multiple unnecessary variable assignments were identified in the strategy implementations, where function parameters are redundantly assigned to local variables with the same name. These assignments do not perform any transformation, modification, or provide any benefit.

Code Location

The following redundant assignments were found across multiple strategies:

- **LayerBank** Strategy

```
40 | let strategy = strategy;
```

- **Echelon** Strategy

```
95 | let strategy = strategy;
```

```
26 | let strategy = strategy;
```

- **Meridian** Strategy

```
59 | let strategy = strategy;
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to remove the aforementioned unnecessary variable assignments. The parameters can be used directly without the redundant assignment, which will improve code clarity and reduce unnecessary lines of code.

Remediation Comment

SOLVED: The **Canopy team** solved this issue in the specified pull request.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/pull/72>

8.14 INEFFICIENT GLOBAL BORROW OF STRATEGY RESOURCE IN REWARDS POOL SETTER

// INFORMATIONAL

Description

The Echelon strategy's `set_rewards_pool_address` function borrows the strategy global resource before checking if `RewardsPoolDetails` exists, but only uses the borrow inside the conditional block when the resource does not exist. This creates unnecessary gas costs from borrowing the reference outside where it's needed.

Code Location

Below is a code snippet of the `set_rewards_pool_address` function:

```
11 | let strategy_ref = borrow_strategy(strategy);
12 | if (!exists<RewardsPoolDetails>(object::object_address(&strategy))) {
13 |     initialize_reward_pool_details(strategy_ref, rewards_pool_address);
14 | };
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to relocate the `borrow_strategy` operation within the conditional block to enhance gas efficiency and prevent unnecessary resource borrowing.

Remediation Comment

SOLVED : The `Canopy team` solved this issue in the specified pull request.

Remediation Hash

<https://github.com/Canopyxyz/satay-movement/pull/72>

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.